

DemandTest: Demand-Driven Static Discovery of Project-Specific Knowledge for Efficient Intention-Aligned Unit Test Generation

Kaleb Guo

University of Illinois Urbana-Champaign

Urbana, IL, USA

TODO@illinois.edu

Abstract

Industrial unit tests are written to validate a requirement—a *validation intention*—not to maximize coverage, and writing such a test requires project-specific knowledge (constructors, factories, fixtures, mocks, assertion idioms) that is scattered across thousands of files. Repository-level code agents find this knowledge by reading files one at a time, which makes them expensive in tokens and wall-clock time; existing context-injection generators retrieve fixed context patterns and have no notion of whether the retrieved context is *sufficient*. We propose DEMANDTEST, which addresses ICSE 2027 Industry Challenge #8. Given a focal method, a Java repository, and a natural-language intention, DEMANDTEST derives a typed *demand set* from the focal signature and the intention, resolves every demand against a one-time static project index using *construction recipes*, expands context only along edges that satisfy an unresolved demand, and stops when a decidable *sufficiency predicate* holds. The LLM then receives one compact knowledge packet and generates the test in a single call; mechanical failures are repaired statically and at most one LLM repair round is allowed. We will evaluate DEMANDTEST against OpenHands on 13 open-source Java projects plus one or two repositories with more than 10,000 source files, using open-weight models, on the challenge’s own metrics: compilation and execution rate, semantic alignment, tokens, response time, and files inspected. The project is at the specification stage; this proposal fixes the design, the evaluation protocol, and the plan.

1 Problem

Background. Coverage is not what developers write tests for. In practice a test validates that a requirement, a feature scenario, or an expected behavior has been implemented; Qi et al. [9] call this a *validation intention* and instantiate it as a short semi-structured description: an *objective* (≤ 50 words), optional *preconditions*, and optional *expected results*. Turning such an intention into an executable unit test is costly in industry because the required project-specific knowledge—which constructor or factory produces a valid input, which fixtures and mocks the project uses, which helper asserts the expected state—is spread over a large codebase and is rarely textually similar to the intention. ICSE 2027 Industry Challenge #8 [1] states the problem precisely: *given a focal method, a large industrial codebase, and a natural-language validation intention, how can we efficiently generate an executable, project-specific unit test that faithfully validates*

the intended behavior? The challenge lists three obstacles—efficient discovery of testing knowledge, assessing whether the retrieved knowledge is *sufficient*, and balancing effectiveness, efficiency, and confidentiality—and five requirements: intention alignment (R1), scalability to repositories with more than 10,000 source files (R2), at least 50% fewer tokens and 20% less wall-clock time than strong repository-agent baselines at comparable quality (R3), deployability with local or private LLMs (R4), and evaluation on compilation rate, execution rate, semantic alignment, developer acceptance, token cost, response time, and number of inspected files (R5).

Why existing approaches fall short. Search-based tools such as EvoSuite [5] maximize structural coverage and cannot target an intention. Prompt-only LLM generators such as ChatTester [15] hallucinate project APIs and miss setup logic. A recent line of work injects project context obtained by static analysis or a language server—KTester [7], RAtester [14], APT [16], CATGen [3]—and IntentionTest [9] retrieves a similar existing test and edits it under the intention with up to four LLM refinement rounds. These systems raise compilation and pass rates, but they either look up context reactively inside a multi-turn loop (RAtester), or inject fixed context patterns with no criterion for when enough has been collected, and none of them is measured against repository agents on the challenge’s efficiency metrics; IntentionTest reports no token or latency numbers at all. Repository agents such as SWE-agent [13] and OpenHands [11] are strong on repository-level test generation [8] but pay for it with long tool-call trajectories, and cost is now a first-class concern in agent design [10]. Agentless [12] showed for program repair that a fixed localize–generate–validate pipeline can rival agents at a fraction of the cost. We ask the analogous question for intention-driven test generation, with one addition the challenge asks for explicitly: a decidable answer to “is the collected knowledge sufficient?”

Problem statement. Design a generator that, for a focal method m in class C of a Java repository R and an intention I , produces a JUnit test that compiles, executes, and validates I , while minimizing LLM tokens, LLM calls, wall-clock time, and the number of repository files inspected, and that runs with open-weight models. We do not study program repair, vulnerability detection, coverage-driven generation, or general-purpose agents, and we never claim to find real bugs: correctness is measured against the real code and against held-out human-written tests.

Table 1: DemandTest stages and the LLM calls each may make.

Stage	What it does	LLM calls
S0	Project index, once per repository	0
S1	Demand set $D(m, I)$ from signature and intention	0
S2	Bounded expansion until Σ holds	0
S3	Generation from one knowledge packet	1
S4	Static repair; compile and run	0
S5	Semantic repair, only on failure	≤ 1

2 Solution

Thesis. Most of what an agent spends tokens on is *finding* testing knowledge. That knowledge is largely determined by types: to call m one needs a receiver of type C and arguments of the parameter types; to set up m one needs the collaborators m reads; to assert one needs an observable of the return or state type. These needs can be computed before any LLM call, resolved against a static index of the project’s construction paths and test idioms, and checked for sufficiency without an LLM. DEMANDTEST is that computation followed by one generation call. Table 1 summarizes the pipeline; the full specification, including the index schema, module contracts, prompts, and acceptance tests, is in the repository (<https://github.com/TODO/demandtest>, [docs/SPEC.md](#)).

S0: Project index. A one-time pass with JavaParser and its symbol solver [2] records, for every type, its constructors, static factories, builders, singletons (including enum constants), public observables (getters, `equals`, `toString`, public fields), the fields each method reads and writes, and, for every existing test, its fixtures, helper methods, resolved callees, and the assertion and mocking libraries it uses. Every entity carries the file it lives in, so file accounting is exact.

S1: Demand set. $D(m, I)$ contains a *receiver* need for C (unless m is static), one *argument* need per parameter, annotated with the sentence of the preconditions that mentions the parameter, one *setup* need per field of C that m reads, *oracle* needs derived from the expected results (an exception need if they mention throwing, otherwise a return-value need and, if they describe state change or m is void, a state need; an interaction need if they mention delegation), and an *idiom* need for the project’s test framework, assertion library, and mocking library. No LLM is involved.

S2: Recipes, sufficiency, bounded expansion. A *recipe* is a way to obtain a value of a type: a JDK literal, a fixture or helper from an existing test, a constructor, a factory, a builder, a singleton, or a mock (only for interfaces and abstract classes when the project already mocks). Its cost is the number of parameters that are not literal-resolvable, summed over the sub-recipes chosen for those parameters; ties prefer how the project already does it (fixture before constructor before factory before mock). Resolution is demand-driven in the sense of demand-driven program analysis [6]: only the types

a need asks for are ever resolved, recursively to a bounded depth, with memoization. The *sufficiency predicate* $\Sigma(D, ctx)$ holds when every receiver, argument, and setup need has a recipe whose entities are in the context ctx , every oracle need has an observable in ctx (or a mock is admissible), and the idiom is known. Expansion starts from the focal signature and, while Σ fails and a file budget B_f (default 12) and a token budget B_t (default 2,000) are not exhausted, picks an unresolved need, resolves it over the whole index, and adds the recipe’s entities and files to ctx . If Σ still fails, DEMANDTEST falls back to IntentionTest-style retrieval of two referable tests and marks the run as *fallback*, so that the predicate’s accuracy can be measured. Every step is logged as a trace.

S3–S5: One call, static repair, one repair call. The context is rendered into a deterministic *knowledge packet*: intention, focal source, one line per resolved need with the recipe as a code fragment and the signatures it depends on, observables for the oracle, project facts, and one assertion-style example. A single temperature-0 call produces the test class; a parse failure counts as a failed run, with no retry. Static repair then fixes the package declaration (which also grants package-private access), missing imports resolved through the index, missing `throws` clauses, and JUnit-4/5 idiom mismatches—following the observation of CATGen [3] that program-analysis post-processing removes most iterative LLM repair; we adopt this step and do not claim it. Only if the test compiles but fails, or javac diagnostics survive, is one repair call made with the trimmed diagnostics and the packet, opening no new files.

Harness and deployability. All systems, including baselines, write to one SQLite ledger (`llm_calls`, `file_access`, `results`); every number reported is a query over it. The LLM client is an OpenAI-compatible HTTP call with no other dependency, so open-weight models on a private endpoint satisfy R4. Runs are idempotent per task and configuration and resume after interruption.

What is new. Relative to the closest work, DEMANDTEST contributes (i) a demand set computed from types and the intention *before* any LLM call, rather than reactive lookups or fixed context patterns; (ii) an explicit, decidable sufficiency predicate with bounded expansion, answering the challenge’s second obstacle; and (iii) a comparison against repository agents on the challenge’s efficiency metrics, which no prior test-generation work reports.

3 Evaluation Setup

Subjects. The 13 open-source Java projects of IntentionTest [9] (51–1,099 tests each), for comparability, plus one or two active repositories with more than 10,000 source files (candidates: Elasticsearch, Apache Flink, Spring Framework), chosen after verifying build feasibility, to address R2. No industrial repository is available to us; this is a stated threat to validity, and developer acceptance is approximated by a three-to-five person review.

Tasks and ground truth. Following IntentionTest, an intention is synthesized by an LLM from each existing test that exercises exactly one focal method (objective ≤ 50 words, preconditions plus expected results ≤ 200 words, under 5% code tokens); 10% are checked by two people and the rubric is refined until they agree. The reference test is removed from the index, the test corpus, and the baseline’s checkout while its task runs, so no system sees the answer. Trivial focal methods (getters, setters, ≤ 3 LOC) are dropped.

Baselines and ablations. The primary baseline, required by R3, is OpenHands [11] in headless mode with the same model, the intention and focal location as the task, a 30-iteration cap, three runs per task; its trajectories are parsed into the same ledger. Secondary baselines are an IntentionTest-style retrieve-and-edit pipeline with four refinement rounds re-implemented on our harness, and, time permitting, SWE-agent [13]. Ablations replace the packet with the full focal class, replace demand-driven expansion with embedding retrieval, replace static repair with LLM repair, and sweep B_f , B_t , and the resolution depth.

Models. Main experiments use open-weight models (Qwen3-Coder or DeepSeek-V3 class) through an OpenAI-compatible endpoint, which is what R4 requires; a closed model is run on a subset as an upper-bound reference.

Metrics (R5). Compilation rate; execution (pass) rate; mutation score with PIT [4] on a sampled subset; semantic alignment with the intention by an LLM judge on a 0–2 rubric, calibrated against the human sample (reporting agreement); prompt and completion tokens as reported by the provider (never estimated); number of LLM calls; wall-clock time including compilation and execution; number of inspected files (for the agent, every file a tool call reads or greps into; for DEMANDTEST, the files whose entities enter the packet, with the one-time index reported separately as an amortized cost); and cost per passing test.

Research questions. RQ1: Is test quality comparable to the agent baseline? RQ2: Do tokens, wall-clock time, and inspected files drop past the R3 thresholds? RQ3: Which component matters (ablations)? RQ4: How sensitive are results to model size and to objective-only intentions? RQ5: How accurate is Σ —when it reports *sufficient*, how often does the single call succeed without repair, compared with *fallback* runs?

4 Result Analysis

There are no experimental results yet; this section reports what is in place and how the results will be read.

Current status. The challenge and the closest prior work have been analyzed and the design fixed in a specification (docs/SPEC.md) that defines the demand set, recipes, the sufficiency predicate, the expansion algorithm, the packet format, the fixed prompts, the `index.json` schema, the ledger schema, the CLI, the task-construction and leakage-control procedure, the baseline protocol, and per-module acceptance

tests that run without an LLM or a Java toolchain. The ledger schema is frozen so that the indexer, the pipeline, and the baseline tracks can proceed independently. Pilot subjects (two small IntentionTest projects and one large repository) are being set up.

How results will be read. The headline artifact is a Pareto plot of pass rate against tokens per task, with one point per system and per DEMANDTEST configuration, and a table of the R5 metrics per system. The R3 thresholds (50% tokens, 20% wall-clock) are the pass/fail criterion for the project; inspected files is expected to be the metric where the gap is largest, since DEMANDTEST opens files only when a need requires them. We anticipate three failure modes and will report them rather than hide them: Σ may hold while the single call still fails (measured by RQ5); setup needs may over-approximate on large classes (capped at five fields, ablated); and large repositories may not build cleanly (we will report the subset that does).

5 Future Plan

The course project concludes in mid-November, aligned with the challenge’s solution-paper deadline (November 13, 2026). The next month is the critical path.

Weeks 1–2 (through October 1). Implement S0 (indexer) and S1–S2 with their acceptance tests on two pilot projects; implement the ledger, the accounting client, and the resumable CLI; run OpenHands on ten pilot tasks and parse its trajectories, so that both sides of the comparison exist early.

Weeks 3–4 (through October 15). End-to-end runs on the pilots with an open-weight model; first ablation (packet vs. full class, static vs. LLM repair); a go/no-go check against R3 on the pilots. If the thresholds are not met, the first levers are the packet budget and the recipe ranking, both of which are configuration rather than design changes.

October 16 – November 13. Scale to all 13 projects and the large repository; mutation testing on the sampled subset; LLM-judge calibration and the human review; SWE-agent if time permits; analysis, figures, and the write-up. Collaborators from the lab take the indexer, the harness, and the baseline-and-evaluation tracks respectively; a weekly sync reviews the ledger.

References

- [1] 2026. Efficient Project-Specific Test Generation with Validation Intentions. ICSE 2027 Industry Challenge, accepted challenge #8. <https://conf.researchr.org/track/icse-2027/icse-2027-industry-challenge>.
- [2] 2026. JavaParser: Java 1–21 Parser and Abstract Syntax Tree for Java. <https://javaparser.org>.
- [3] Junjie Chen, Ziqi Wang, Lin Yang, Chen Yang, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, and Dong Wang. 2026. Context Matters: Improving the Practical Reliability of LLM-Based Unit Test Generation. *arXiv preprint arXiv:2607.19682* (2026).
- [4] Henry Coles, Thomas Laurent, Christopher Henard, Mike Papadakis, and Anthony Ventresque. 2016. PIT: A Practical Mutation Testing Tool for Java. In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA)*.

- [5] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic Test Suite Generation for Object-Oriented Software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*.
- [6] Nevin Heintze and Olivier Tardieu. 2001. Demand-Driven Pointer Analysis. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- [7] Anji Li, Mingwei Liu, Zhenxi Chen, Zheng Pei, Zike Li, Dekun Dai, Yanlin Wang, and Zibin Zheng. 2026. Knowledge Matters: Injecting Project and Testing Knowledge into LLM-based Unit Test Generation. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*.
- [8] Niels Mündler, Mark Niklas Mueller, Jingxuan He, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [9] Binhang Qi, Yun Lin, Xinyi Weng, Yuhuan Huang, Chenyan Liu, Hailong Sun, Zhi Jin, and Jin Song Dong. 2025. Intention-Driven Generation of Project-Specific Test Cases. *arXiv preprint arXiv:2507.20619* (2025).
- [10] Ningning Wang, Xavier Hu, Pai Liu, He Zhu, Yue Hou, Heyuan Huang, Shengyu Zhang, Jian Yang, Jiaheng Liu, Ge Zhang, Changwang Zhang, Jun Wang, Yuchen Eleanor Jiang, and Wangchunshu Zhou. 2025. Efficient Agents: Building Effective Agents While Reducing Cost. *arXiv preprint arXiv:2508.02694* (2025).
- [11] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations (ICLR)*.
- [12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025).
- [13] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [14] Xin Yin, Chao Ni, Xinrui Li, Liushan Chen, Guojun Ma, and Xiaohu Yang. 2025. Enhancing LLM’s Ability to Generate More Repository-Aware Unit Tests Through Precise Contextual Information Injection. *arXiv preprint arXiv:2501.07425* (2025).
- [15] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024).
- [16] Zhe Zhang, Xingyu Liu, Yuanzhang Lin, Xiang Gao, Hailong Sun, and Yuan Yuan. 2024. LLM-based Unit Test Generation via Property Retrieval. *arXiv preprint arXiv:2410.13542* (2024).